

Week 5: Payment Transaction Management System

Scenario:

The e-commerce company needs to maintain payment information for customer orders. The system should record payment details, track transaction status, manage successful and failed payments, and generate payment-related business reports.

Students will create a **Payment Transaction Module** connected with the existing Customer and Order Management modules.

Tasks to be Done:

1. Create Payment Table

Payment Entity

Stores customer payment transaction details. Attribute	Description
Payment_ID	Unique payment identifier
Order_ID	Order reference
Payment_Date	Date of transaction
Payment_Mode	UPI/Card/Net Banking/Cash
Payment_Status	Success/Failed/Pending
Transaction_Amount	Paid amount

```
SQL> CREATE TABLE Payment(  
  2 Payment_ID VARCHAR2(7),  
  3 OrderID VARCHAR2(7),  
  4 PaymentDate DATE,  
  5 PaymentMode VARCHAR2(15),  
  6 PaymentStatus VARCHAR2(10),  
  7 TransactionAmount NUMBER(10,2),  
  8 PRIMARY KEY (Payment_ID),  
  9 FOREIGN KEY(OrderID) REFERENCES Orders(OrderID));
```

Table created.

3. Store Payment Details

```
SQL> INSERT INTO PAYMENT  
  2 (PAYMENT_ID, ORDERID, PAYMENTDATE, PAYMENTMODE, PAYMENTSTATUS, TRANSACTIONAMOUNT)  
  3 VALUES  
  4 ('PAY0001', 'ORD0001', DATE '2026-01-10', 'Credit Card', 'Completed', 55850.00);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PAYMENT  
  2 (PAYMENT_ID, ORDERID, PAYMENTDATE, PAYMENTMODE, PAYMENTSTATUS, TRANSACTIONAMOUNT)  
  3 VALUES  
  4 ('PAY0002', 'ORD0002', DATE '2026-01-25', 'UPI', 'Completed', 3350.00);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PAYMENT  
  2 (PAYMENT_ID, ORDERID, PAYMENTDATE, PAYMENTMODE, PAYMENTSTATUS, TRANSACTIONAMOUNT)  
  3 VALUES  
  4 ('PAY0003', 'ORD0004', DATE '2026-02-18', 'Debit Card', 'Completed', 8300.00);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PAYMENT  
  2 (PAYMENT_ID, ORDERID, PAYMENTDATE, PAYMENTMODE, PAYMENTSTATUS, TRANSACTIONAMOUNT)  
  3 VALUES  
  4 ('PAY0004', 'ORD0006', DATE '2026-03-15', 'Net Banking', 'Completed', 7000.00);
```

1 row created.

```
SQL> SELECT * FROM Payment;
```

PAYMENT	ORDERID	PAYMENTDA	PAYMENTMODE	PAYMENTSTA	TRANSACTIONAMOUNT
PAY0001	ORD0001	10-JAN-26	Credit Card	Completed	55850
PAY0002	ORD0002	25-JAN-26	UPI	Completed	3350
PAY0003	ORD0004	18-FEB-26	Debit Card	Completed	8300
PAY0004	ORD0006	15-MAR-26	Net Banking	Completed	7000
PAY0006	ORD0010	18-MAY-26	UPI	Completed	5200
PAY0007	ORD0012	20-JUN-26	Credit Card	Completed	2450
PAY0008	ORD0014	15-AUG-26	Debit Card	Completed	28000
PAY0009	ORD0014	14-AUG-26	Debit Card	Failed	28000
PAY0005	ORD0008	20-APR-26	COD	Completed	5700

```
9 rows selected.
```

4. Manage Successful and Failed Transactions

Tasks:

Write SQL queries to:

1. Display all successful payments.
2. Find failed transactions.
3. Count total successful and failed payments.

```
SQL> SELECT * FROM Payment WHERE PaymentStatus = 'Completed';
```

PAYMENT	ORDERID	PAYMENTDA	PAYMENTMODE	PAYMENTSTA	TRANSACTIONAMOUNT
PAY0001	ORD0001	10-JAN-26	Credit Card	Completed	55850
PAY0002	ORD0002	25-JAN-26	UPI	Completed	3350
PAY0003	ORD0004	18-FEB-26	Debit Card	Completed	8300
PAY0004	ORD0006	15-MAR-26	Net Banking	Completed	7000
PAY0006	ORD0010	18-MAY-26	UPI	Completed	5200
PAY0007	ORD0012	20-JUN-26	Credit Card	Completed	2450
PAY0008	ORD0014	15-AUG-26	Debit Card	Completed	28000
PAY0005	ORD0008	20-APR-26	COD	Completed	5700

```
8 rows selected.
```

```
SQL> SELECT * FROM Payment WHERE PaymentStatus = 'Failed';
```

PAYMENT	ORDERID	PAYMENTDA	PAYMENTMODE	PAYMENTSTA	TRANSACTIONAMOUNT
PAY0009	ORD0014	14-AUG-26	Debit Card	Failed	28000

```
SQL> SELECT PaymentStatus, COUNT(PaymentStatus) FROM Payment GROUP BY PaymentStatus  
2 ;
```

PAYMENTSTA	COUNT(PAYMENTSTATUS)
Completed	8
Failed	1

5. Analyze Payment Methods Used by Customers

Generate payment analysis reports.

Report 1: Payment Mode Analysis

Find:

- Number of transactions using UPI.
- Number of card payments.
- Most preferred payment method.

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS
 2  FROM PAYMENT
 3  WHERE PAYMENTMODE = 'UPI';

UPI_TRANSACTIONS
-----
                2

SQL> SELECT COUNT(*) AS CARD_PAYMENTS
 2  FROM PAYMENT
 3  WHERE PAYMENTMODE IN ('Credit Card', 'Debit Card');

CARD_PAYMENTS
-----
                5

SQL> SELECT
 2     PAYMENTMODE,
 3     COUNT(*) AS TRANSACTION_COUNT
 4  FROM PAYMENT
 5  GROUP BY PAYMENTMODE
 6  ORDER BY TRANSACTION_COUNT DESC;

PAYMENTMODE      TRANSACTION_COUNT
-----
Debit Card                3
UPI                      2
Credit Card             2
COD                     1
Net Banking             1

SQL> SELECT PAYMENTMODE, COUNT(*) AS TRANSACTION_COUNT
 2  FROM PAYMENT
 3  GROUP BY PAYMENTMODE
 4  ORDER BY TRANSACTION_COUNT DESC
 5  FETCH FIRST 1 ROW ONLY;

PAYMENTMODE      TRANSACTION_COUNT
-----
Debit Card                3
```

Report 2: Revenue Analysis

Find:

- Total revenue generated.
- Revenue by payment method.
- Average transaction amount.

```
SQL> SELECT PAYMENTMODE, COUNT(*) AS TRANSACTION_COUNT
2 FROM PAYMENT
3 GROUP BY PAYMENTMODE
4 ORDER BY TRANSACTION_COUNT DESC
5 FETCH FIRST 1 ROW ONLY;
```

PAYMENTMODE	TRANSACTION_COUNT
Debit Card	3

```
SQL> SELECT
2 SUM(TRANSACTIONAMOUNT) AS TOTAL_REVENUE
3 FROM PAYMENT
4 WHERE PAYMENTSTATUS = 'Completed';
```

TOTAL_REVENUE
115850

```
SQL> SELECT
2 PAYMENTMODE,
3 SUM(TRANSACTIONAMOUNT) AS TOTAL_REVENUE
4 FROM PAYMENT
5 WHERE PAYMENTSTATUS = 'Completed'
6 GROUP BY PAYMENTMODE
7 ORDER BY TOTAL_REVENUE DESC;
```

PAYMENTMODE	TOTAL_REVENUE
Credit Card	58300
Debit Card	36300
UPI	8550
Net Banking	7000
COD	5700

```
SQL> SELECT
2 ROUND(AVG(TRANSACTIONAMOUNT), 2) AS AVERAGE_TRANSACTION_AMOUNT
3 FROM PAYMENT
4 WHERE PAYMENTSTATUS = 'Completed';
```

AVERAGE_TRANSACTION_AMOUNT
14481.25

Report 3: Customer Payment History

Display:

- Customer name
- Order ID
- Payment mode
- Amount
- Payment status

```
SQL> SELECT
  2     C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
  3     O.ORDERID AS ORDER_ID,
  4     O.ORDERDATE AS ORDER_DATE,
  5     P.PAYMENTDATE AS PAYMENT_DATE,
  6     P.PAYMENTMODE AS PAYMENT_MODE,
  7     P.TRANSACTIONAMOUNT AS AMOUNT,
  8     P.PAYMENTSTATUS AS PAYMENT_STATUS
  9 FROM CUSTOMER C
 10 JOIN ORDERS O
 11     ON C.CUSTOMER_ID = O.CUSTOMERID
 12 JOIN PAYMENT P
 13     ON O.ORDERID = P.ORDERID
 14 ORDER BY P.PAYMENTDATE;
```

CUSTOMER_NAME			ORDER_I	ORDER_DAT	PAYMENT_D
Arjun Kumar			ORD0001	10-JAN-26	10-JAN-26
Credit Card	55850	Completed			
Priya Sharma			ORD0002	25-JAN-26	25-JAN-26
UPI	3350	Completed			
Ananya Iyer			ORD0004	18-FEB-26	18-FEB-26
Debit Card	8300	Completed			

CUSTOMER_NAME			ORDER_I	ORDER_DAT	PAYMENT_D
Meera Krishnan			ORD0006	15-MAR-26	15-MAR-26
Net Banking	7000	Completed			
Kavya Rajan			ORD0008	20-APR-26	20-APR-26
COD	5700	Completed			
Divya Pillai			ORD0010	18-MAY-26	18-MAY-26
UPI	5200	Completed			

CUSTOMER_NAME			ORDER_I	ORDER_DAT	PAYMENT_D
Sneha Patel			ORD0012	20-JUN-26	20-JUN-26
Credit Card	2450	Completed			
Lakshmi Venkatesan			ORD0014	15-AUG-26	14-AUG-26
Debit Card	28000	Failed			
Lakshmi Venkatesan			ORD0014	15-AUG-26	15-AUG-26
Debit Card	28000	Completed			

Payment Analysis:

```
SQL> SELECT
  2     PAYMENTMODE,
  3     COUNT(*) AS TRANSACTION_COUNT,
  4     SUM(TRANSACTIONAMOUNT) AS TOTAL_REVENUE,
  5     ROUND(AVG(TRANSACTIONAMOUNT), 2) AS AVG_TRANSACTION
  6 FROM PAYMENT
  7 WHERE PAYMENTSTATUS = 'Completed'
  8 GROUP BY PAYMENTMODE
  9 ORDER BY TOTAL_REVENUE DESC;
```

PAYMENTMODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
-----	-----	-----	-----
Credit Card	2	58300	29150
Debit Card	2	36300	18150
UPI	2	8550	4275
Net Banking	1	7000	7000
COD	1	5700	5700